

## AccuKnox DevOps Assignment

### Candidate Details

- Name: Sai Chowdary Kukkapalli
  - Role: DevOps Engineer (Fresher)
- 

### Problem Statement 1: Containerization, Kubernetes & CI/CD

#### Project Overview

Wisecow is a shell-based web application containerized using Docker and deployed on Kubernetes using Minikube.

The application was:

- Dockerized using Dockerfile
  - Pushed to Docker Hub
  - Deployed on Kubernetes
  - Exposed using Kubernetes Service
  - Secured using TLS and Ingress
  - Automated using GitHub Actions CI/CD
- 

#### Docker Image

Docker image was built and pushed to Docker Hub.

#### Build Image

```
docker build -t saichowdary009/wisecow-app:v1 .
```

#### Push Image

```
docker push saichowdary009/wisecow-app:v1
```

---

#### Kubernetes Deployment

Resources created:

- Deployment
- Service
- Ingress
- TLS Secret

Files:

k8s/deployment.yaml

k8s/service.yaml

### **Deploy Application**

kubectl apply -f k8s/

### **Verify Deployment**

kubectl get pods

kubectl get svc

kubectl get ingress

---

### **TLS Configuration**

Self-signed TLS certificates were generated and configured using Kubernetes Secrets.

kubectl create secret tls wisecom-tls \

--cert=tls.crt \

--key=tls.key

Ingress was configured to serve HTTPS traffic.

---

### **CI/CD Pipeline**

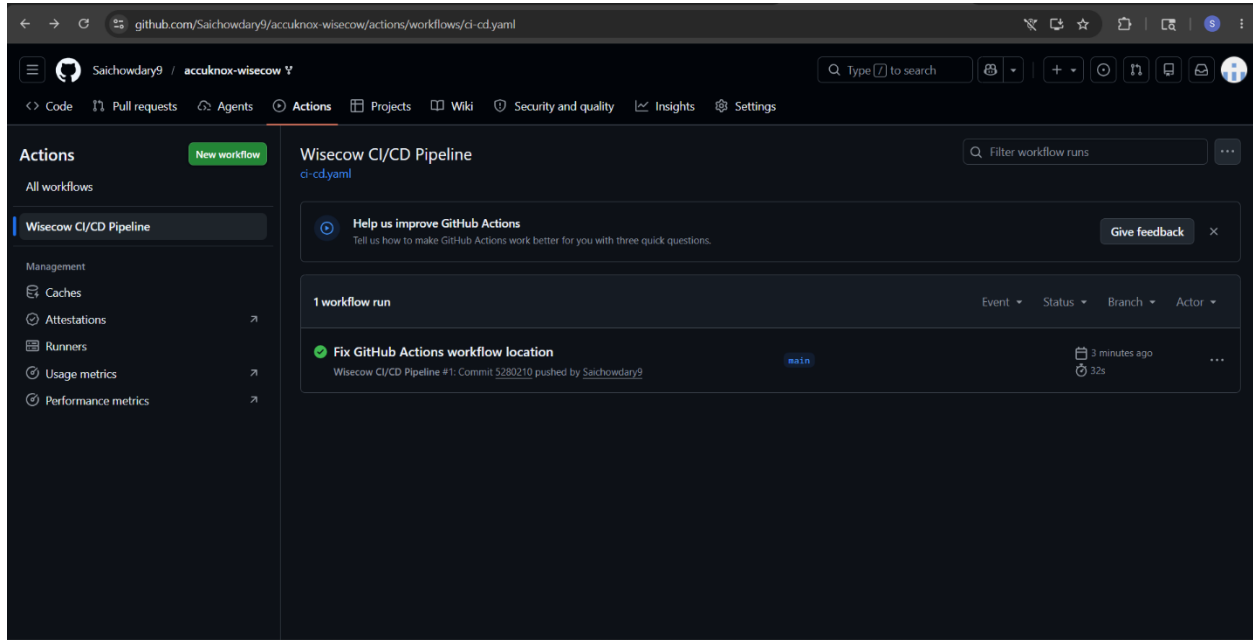
GitHub Actions workflow automatically:

1. Builds Docker image
2. Pushes image to Docker Hub
3. Verifies deployment configuration

Workflow file:

.github/workflows/ci-cd.yaml

GitHub Actions Workflow:

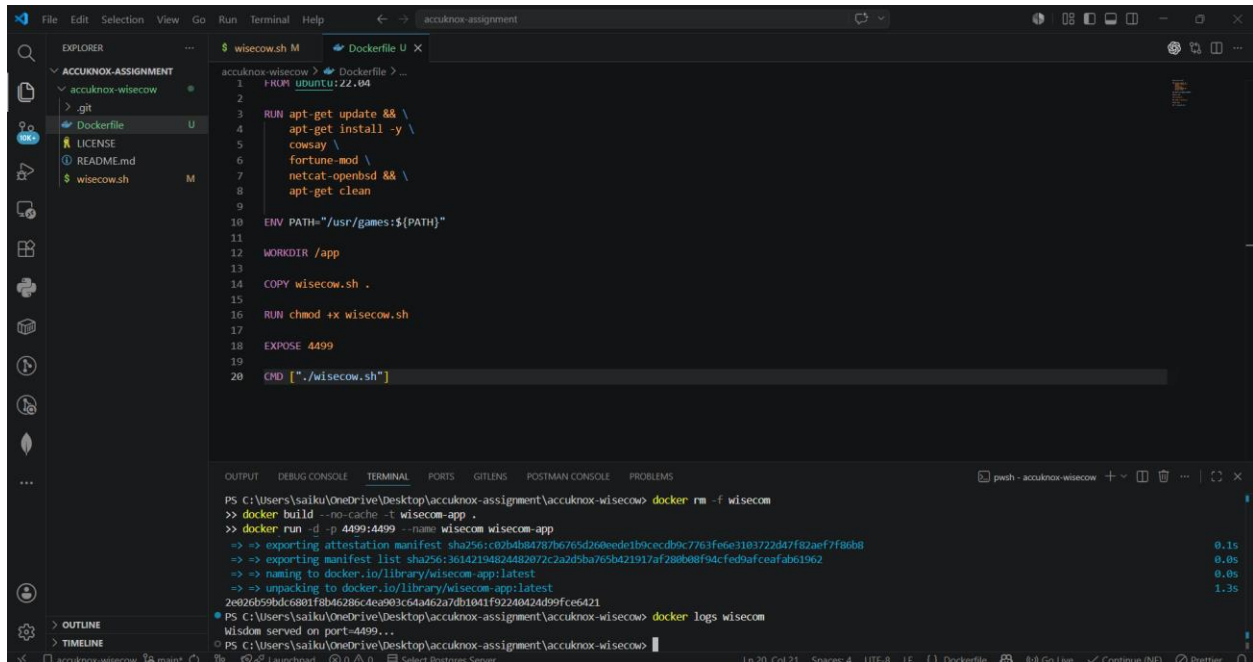


Screenshots:

Docker Output:



Execution:



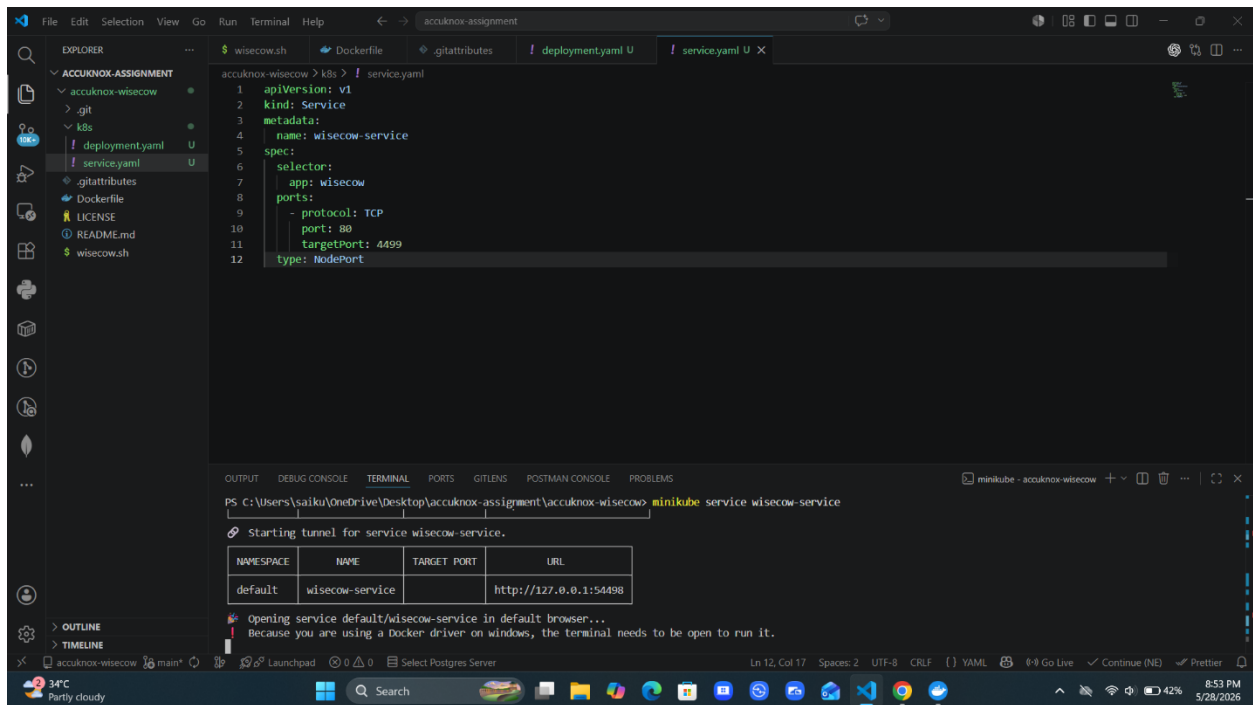
The screenshot shows a VS Code editor with a file explorer on the left displaying the project structure for 'accuknox-wisecow'. The main editor shows a Dockerfile with the following content:

```
1 FROM ubuntu:22.04
2
3 RUN apt-get update && \
4     apt-get install -y \
5         coway \
6         fortune-mod \
7         netcat-openbsd && \
8         apt-get clean
9
10 ENV PATH="/usr/games:${PATH}"
11
12 WORKDIR /app
13
14 COPY wisecow.sh .
15
16 RUN chmod +x wisecow.sh
17
18 EXPOSE 4499
19
20 CMD ["/wisecow.sh"]
```

The terminal at the bottom shows the execution of the following commands:

```
PS C:\Users\saiiku\OneDrive\Desktop\accuknox-assignment\accuknox-wisecow> docker rm -f wisecow
>> docker build --no-cache -t wisecow-app .
>> docker run -d -p 4499:4499 --name wisecow wisecow-app
-> exporting attestation manifest sha256:c02b084787b6765d268e4e1b9ccdb9c7763f6e318372d47f82anf7f8db8
-> exporting manifest list sha256:36142194824482872c2a2d5ba765b421917af288b88f94cfed9afceafab61962
-> naming to docker.io/library/wisecow-app:latest
-> unpacking to docker.io/library/wisecow-app:latest
2e026b59bdc6801f8b46286c4ea903c64a462a7db1041f92240424d99fce6421
PS C:\Users\saiiku\OneDrive\Desktop\accuknox-assignment\accuknox-wisecow> docker logs wisecow
Wisdom served on port=4499...
```

Kubernetes Cluster Output:



The screenshot shows a VS Code editor with a file explorer on the left displaying the project structure for 'accuknox-wisecow'. The main editor shows the 'service.yaml' file with the following content:

```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: wisecow-service
5 spec:
6   selector:
7     app: wisecow
8   ports:
9     - protocol: TCP
10       port: 80
11       targetPort: 4499
12   type: NodePort
```

The terminal at the bottom shows the execution of the following command:

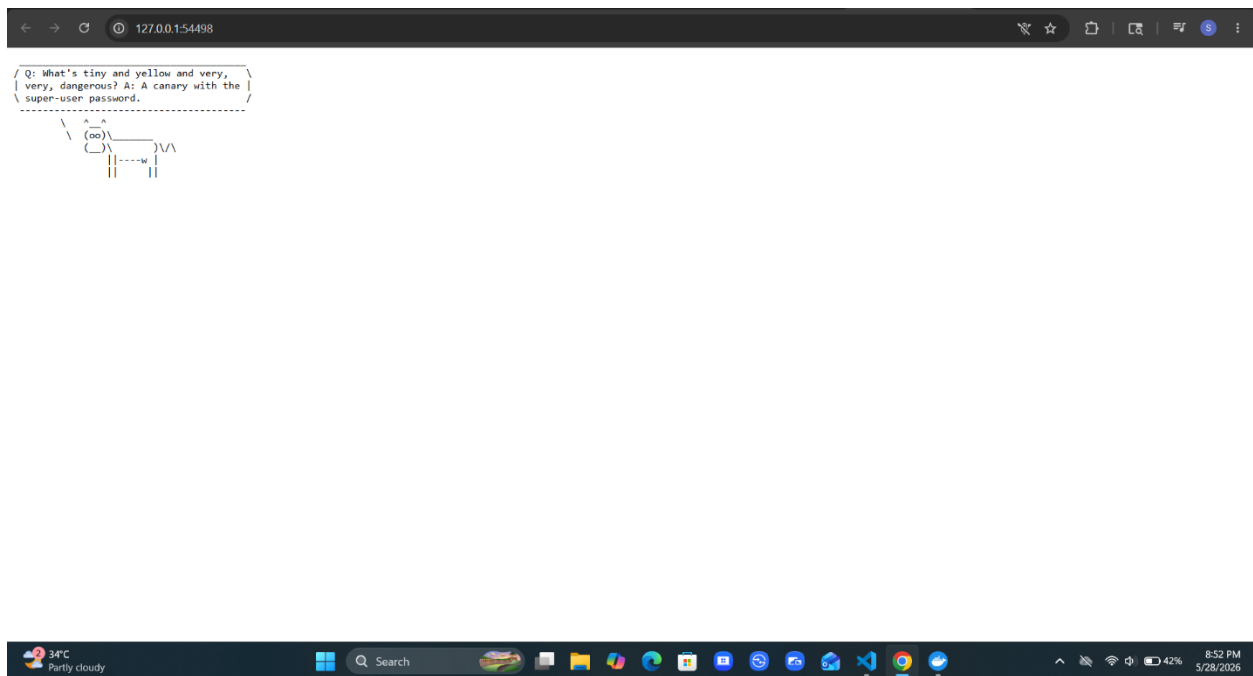
```
PS C:\Users\saiiku\OneDrive\Desktop\accuknox-assignment\accuknox-wisecow> minikube service wisecow-service
```

The output of the command is as follows:

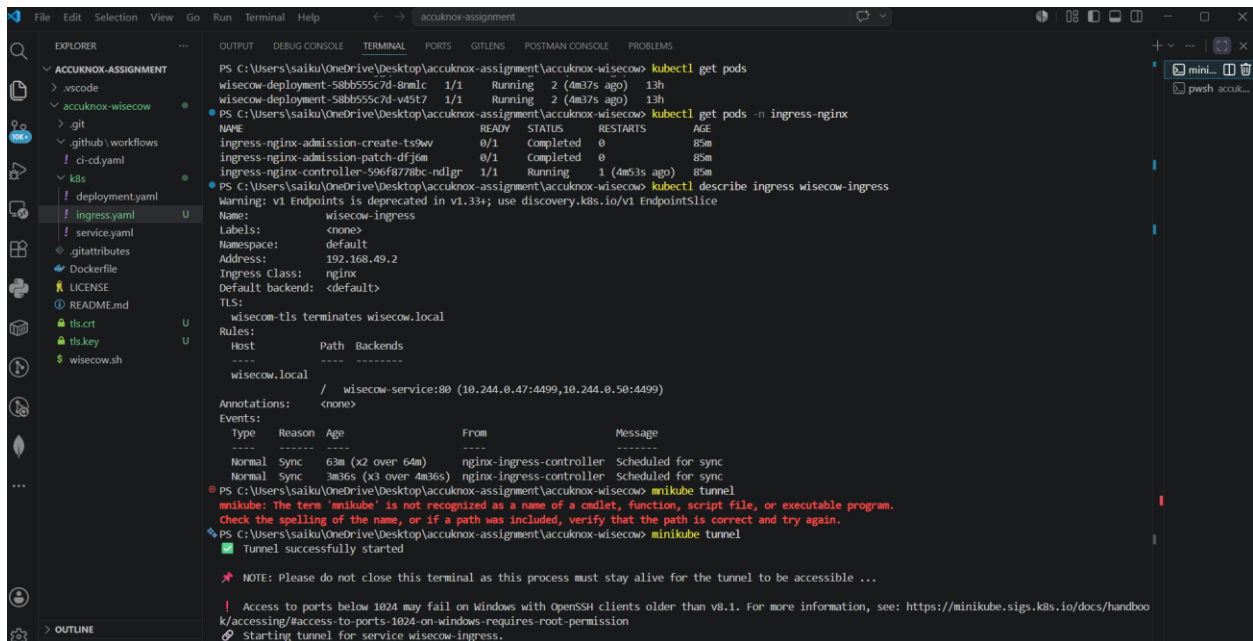
```
Starting tunnel for service wisecow-service.
+-----+-----+-----+-----+
| NAMESPACE | NAME      | TARGET PORT | URL                |
+-----+-----+-----+-----+
| default   | wisecow-service | 80          | http://127.0.0.1:54498 |
+-----+-----+-----+-----+
```

The terminal also shows the following messages:

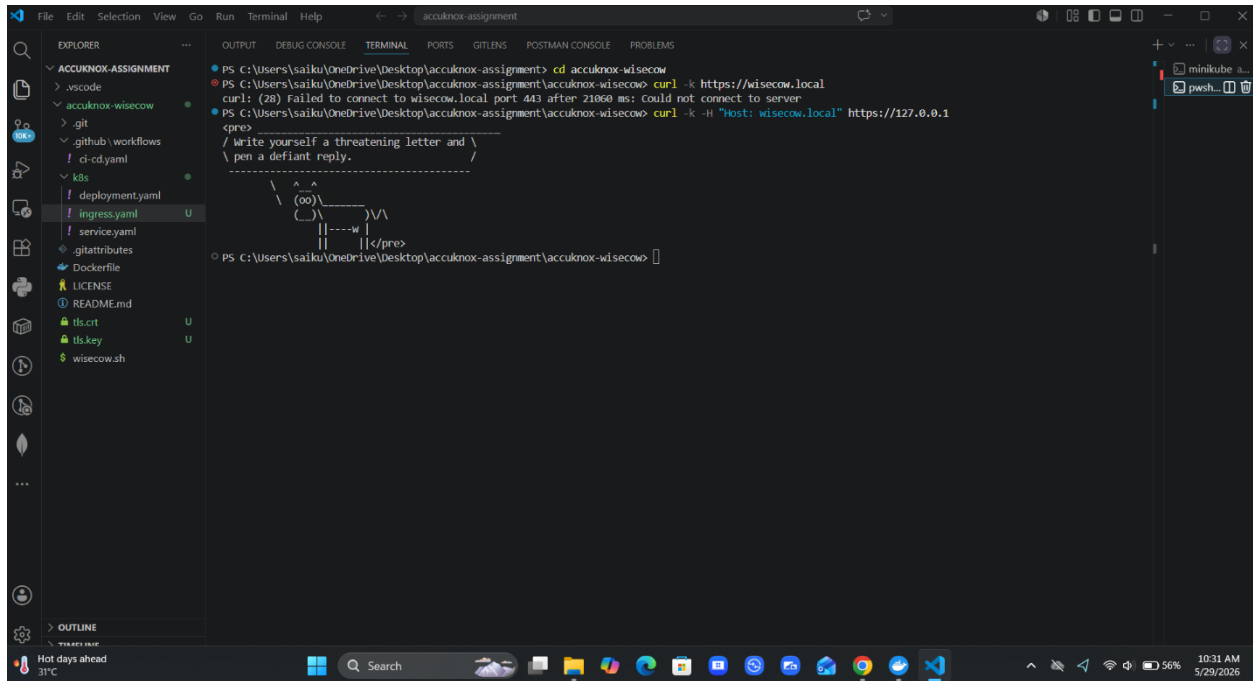
```
Opening service default/wisecow-service in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```



TLS:



Output:



```
PS C:\Users\sai\OneDrive\Desktop\accuknox-assignment> cd accuknox-wisecow
PS C:\Users\sai\OneDrive\Desktop\accuknox-assignment\accuknox-wisecow> curl -k https://wisecow.local
curl: (28) Failed to connect to wisecow.local port 443 after 21060 ms: could not connect to server
PS C:\Users\sai\OneDrive\Desktop\accuknox-assignment\accuknox-wisecow> curl -k -H "Host: wisecow.local" https://127.0.0.1

```

## Problem Statement 2: Monitoring & Health Check Scripts

### System Health Monitoring Script

File:

scripts/system\_monitor.sh

Features:

- CPU usage monitoring
- Memory usage monitoring
- Disk usage monitoring
- Top CPU-consuming processes

Run:

chmod +x system\_monitor.sh

`./system_monitor.sh`

---

## **Application Health Checker**

File:

`scripts/health_check.sh`

Features:

- Checks application availability
- Returns HTTP response code
- Displays UP/DOWN status

Run:

`chmod +x health_check.sh`

`./health_check.sh https://google.com`

Example Output:

Application Status: UP

HTTP Status Code: 301

Output:

Health\_monitor

```
MINGW64/C:/Users/saiku/OneDrive/Desktop/accuknox-assignment/accuknox-wisecow/scripts
saiku@LAPTOP-43K4ZE5C MINGW64 ~ (main)
$ cd /c:/Users/saiku/OneDrive/Desktop/accuknox-assignment/accuknox-wisecow/script
bash: cd: /c:/Users/saiku/OneDrive/Desktop/accuknox-assignment/accuknox-wisecow/s
cript: No such file or directory
saiku@LAPTOP-43K4ZE5C MINGW64 ~ (main)
$ AC
saiku@LAPTOP-43K4ZE5C MINGW64 ~ (main)
$ cd /c:/Users/saiku/OneDrive/Desktop/accuknox-assignment/accuknox-wisecow/script
$
saiku@LAPTOP-43K4ZE5C MINGW64 ~/OneDrive/Desktop/accuknox-assignment/accuknox-wisecow/scripts (main)
$ ls
health_check.sh* system_monitor.sh*
saiku@LAPTOP-43K4ZE5C MINGW64 ~/OneDrive/Desktop/accuknox-assignment/accuknox-wisecow/scripts (main)
$ ./health_check.sh https://google.com
Application Status: UP
HTTP Status Code: 301
saiku@LAPTOP-43K4ZE5C MINGW64 ~/OneDrive/Desktop/accuknox-assignment/accuknox-wisecow/scripts (main)
$ ./health_check.sh https://google123.com
Application Status: DOWN
HTTP Status Code: 403
saiku@LAPTOP-43K4ZE5C MINGW64 ~/OneDrive/Desktop/accuknox-assignment/accuknox-wisecow/scripts (main)
$ |
```

System\_monitor Output:

```
File Edit Selection View Go Run Terminal Help
accuknox-assignment
EXPLORER
  ACCUKNOX-ASSIGNMENT
  .vscode
  .accuknox-wisecow
    .git
    .github
    .k8s
    scripts
      health_check.sh
      system_monitor.sh
      .gitattributes
    Dockerfile
    kubearmor-policy.yaml
  LICENSE
  README.md
  tls.crt
  tls.key
  wisecow.sh
  .gitignore
  OUTLINE
  TRACES
  30°C Partly sunny
  Search
  1:22 PM 5/29/2026

accuknox-wisecow > kubearmor-policy.yaml
You 18 minutes ago | author (You)
OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLINS POSTMAN CONSOLE PROBLEMS
PS C:\Users\saiiku\OneDrive\Desktop\accuknox-assignment> minikube ssh
CPU Usage:
%cpu(s): 0.0 us, 11.5 sy, 0.0 ni, 88.5 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
Memory Usage:
total used free shared buff/cache available
Mem: 7.6Gi 2.4Gi 2.8Gi 14Mi 2.6Gi 5.2Gi
Swap: 2.8Gi 0 2.8Gi
Disk Usage:
Filesystem Size Used Avail Use% Mounted on
overlay 100Gi 286 9296 3% /
Top 5 Processes:
USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
root 2119 7.4 3.7 1517724 296684 ? Ssl 05:52 8:57 kube-apiserver --advertise-address=192.168.49.2 --allow-privileged=true --authorization-mode=Node,RBAC --client-ca-file=/var/lib/minikube/certs/ca.crt --enable-bootstrap-token-auth=true --etcd-cafile=/var/lib/minikube/certs/etcd/ca.crt --etcd-cert-file=/var/lib/minikube/certs/apiserver-etcd-client.crt --etcd-keyfile=/var/lib/minikube/certs/apiserver-etcd-client.key --etcd-servers=https://127.0.0.1:2379 --kubernetes-certificate=/var/lib/minikube/certs/apiserver-kubelet-client.crt --kubernetes-client-key=/var/lib/minikube/certs/apiserver-kubelet-client.key --kubernetes-preferred-address-types=InternalIP,ExternalIP,Hostname --proxy-client-cert-file=/var/lib/minikube/certs/front-proxy-client.crt --proxy-client-key-file=/var/lib/minikube/certs/front-proxy-client.key --requestheader-allowed-names=front-proxy-client --requestheader-client-ca-file=/var/lib/minikube/certs/front-proxy-ca.crt --requestheader-extra-headers-prefix=X-Remote-Extra --requestheader-group-headers=X-Remote-Group --requestheader-username-headers=X-Remote-User --secure-port=8443 --service-account-issuer=https://kubernetes.default.svc.cluster.local --service-account-key-file=/var/lib/minikube/certs/sa.pub --service-account-signing-key-file=/var/lib/minikube/certs/sa.key --service-cluster-ip-range=10.96.0.0/12 --tls-cert-file=/var/lib/minikube/certs/apiserver.crt --tls-private-key-file=/var/lib/minikube/certs/apiserver.key --enable-admission-plugins=NamespaceLifecycle,LimitRanger,ServiceAccount,DefaultStorageClass,DefaultTolerationSeconds,NodeRestriction,MutatingAdmissionWebhook,ValidatingAdmissionWebhook,ResourceQuota
root 1407 4.4 1.1 2692020 94688 ? Ssl 05:52 5:24 /var/lib/minikube/binaries/v1.35.1/kubelet --bootstrap-kubeconfig=/etc/kubernetes/bootstrap-kubelet.conf --config=/var/lib/kubelet/config.yaml --hostname-override=minikube --kubernetes-override-minikube --kubeconfig=/etc/kubernetes/kubelet.conf --node-ip=192.168.49.2
root 2063 4.3 1.0 11742224 82636 ? Ssl 05:52 5:11 etcd --advertise-client-urls=https://192.168.49.2:2379 --cert-file=/var/lib/minikube/certs/etcd/server.crt --client-cert-auth=true --data-dir=/var/lib/minikube/etcd --feature-gates=InitialCorruptCheck=true --initial-advertise-peer-urls=https://192.168.49.2:2380 --initial-cluster=minikube=https://192.168.49.2:2380 --key-file=/var/lib/minikube/certs/etcd/server.key --listen-client-urls=https://127.0.0.1:2379,https://192.168.49.2:2379 --listen-metrics-urls=http://127.0.0.1:2381 --listen-peer-urls=https://192.168.49.2:2380 --name=minikube --peer-cert-file=/var/lib/minikube/certs/etcd/peer.crt --peer-client-cert-auth=true --peer-key-file=/var/lib/minikube/certs/etcd/peer.key --peer-trusted-ca-file=/var/lib/minikube/certs/etcd/ca.crt --snapshot-count=10000 --trusted-ca-file=/var/lib/minikube/certs/etcd/ca.crt --watch-progress-notify-interval=5s
root 2065 3.1 1.3 1301552 105936 ? Ssl 05:52 3:50 kube-controller-manager --authentication-kubeconfig=/etc/kubernetes/controller-manager.conf --authorization-kubeconfig=/etc/kubernetes/controller-manager.conf --bind-address=127.0.0.1 --client-ca-file=/var/lib/minikube/certs/ca.crt --cluster-cid=
```



### Problem Statement 3 (Bonus) – KubeArmor

KubeArmor Operator was successfully installed and investigated.

Files:

kubearmor-policy.yaml

A Zero Trust policy was created for the Wisecow workload.

During testing on Minikube (Docker + WSL2), the environment did not expose supported Linux Security Modules (AppArmor/BPF/SELinux), preventing full enforcement validation.

The policy YAML and troubleshooting evidence are included in the repository.

---

### Repository Structure

```
.
├── .github/
│   └── workflows/
│       └── ci-cd.yaml
├── k8s/
│   ├── deployment.yaml
│   └── service.yaml
├── scripts/
│   ├── health_check.sh
│   └── system_monitor.sh
├── kubearmor-policy.yaml
├── Dockerfile
├── wisecow.sh
└── README.md
```

---

Output:

The screenshot shows a Visual Studio Code editor with a project named 'accuknox-assignment'. The Explorer pane on the left shows a file tree with folders like '.vscode', 'accuknox-wisecow', and files like 'kubearmor-policy.yaml'. The main editor area shows the 'kubearmor-policy.yaml' file with the following content:

```
apiVersion: security.kubearmor.com/v1
```

The OUTPUT pane on the right shows the logs for the 'kubearmor' pod. The logs indicate that the pod is running and that the kubearmor operator is successfully installing the kubearmor agent on the node. The logs also show that the kubearmor agent is running and that it is successfully installing the kubearmor policy on the node.

## Technologies Used

- Docker
- Kubernetes
- Minikube
- GitHub Actions
- Docker Hub
- Linux Shell Scripting
- TLS/Ingress
- KubeArmor

## Author

Sai Chowdary Kukkapalli